EXPERIMENT 12: MINIMAX ALGORITHM FOR GAMING

AIM

To implement the **Minimax algorithm** in Python for making optimal decisions in a two-player game.

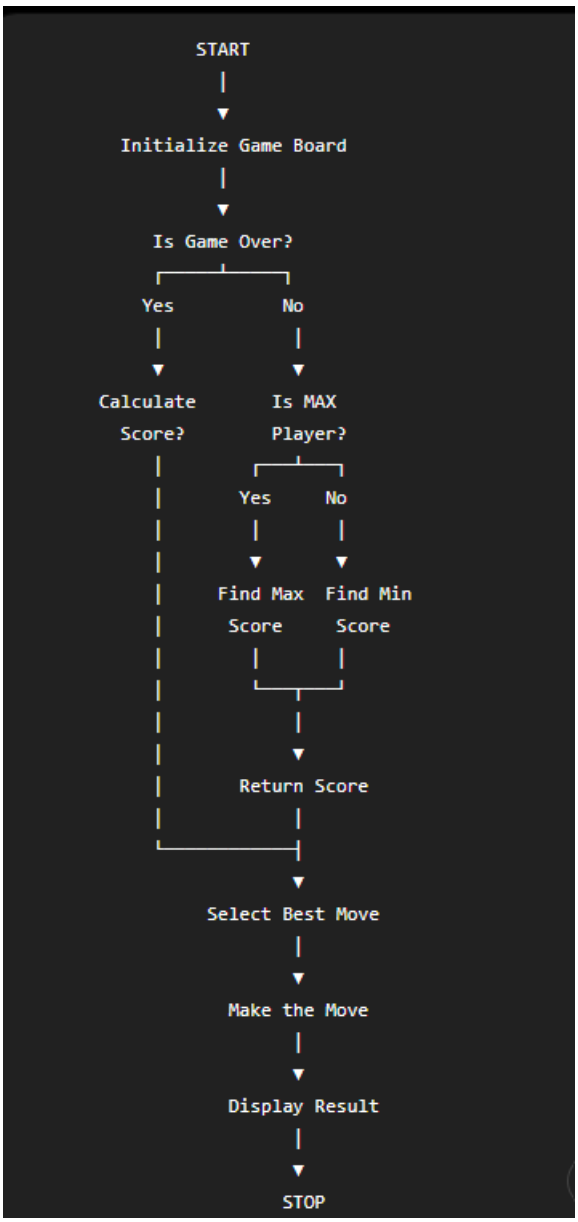
OBJECTIVE

- To understand the Minimax algorithm.
- To implement decision-making for a game.
- To find the best possible move for a player.
- To understand the concepts of **MAX** and **MIN** players.

ALGORITHM

1. Start the program.
2. Define the game board and possible moves.
3. Check whether the game is over.
4. If the current player is **MAX**, select the move with the highest score.
5. If the current player is **MIN**, select the move with the lowest score.
6. Recursively evaluate all possible moves.
7. Return the best score.
8. Select and perform the best move.
9. Display the result.
10. Stop.

FLOWCHART

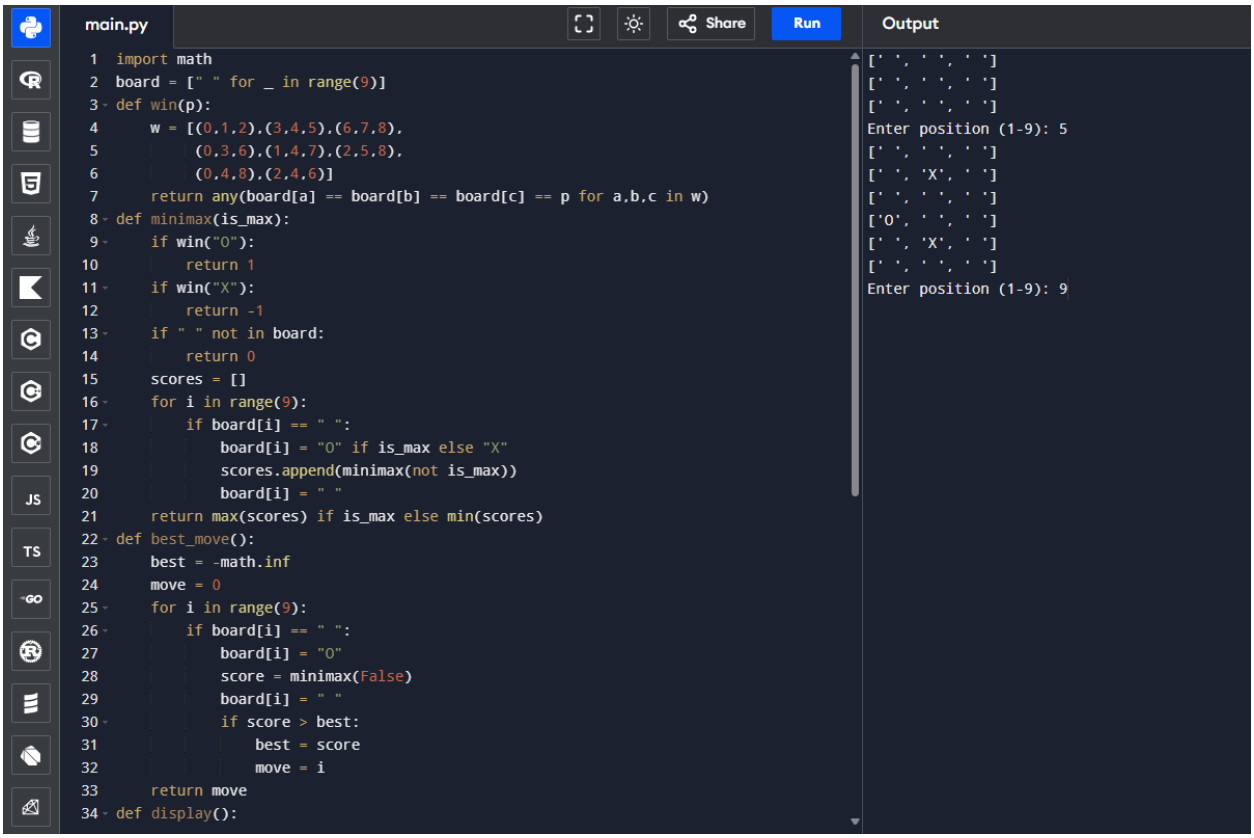

START
Initialize Game Board
Is Game Over?
Yes
No
Calculate Score?
Is MAX Player?
Yes
No
Find Max Score
Find Min Score
Return Score
Select Best Move
Make the Move
Display Result
STOP

PYTHON PROGRAM

```
import math
board = [" " for _ in range(9)]
def win(p):
    w = [(0,1,2),(3,4,5),(6,7,8),
         (0,3,6),(1,4,7),(2,5,8),
         (0,4,8),(2,4,6)]
    return any(board[a] == board[b] == board[c] == p for a,b,c in w)
def minimax(is_max):
    if win("O"):
        return 1
    if win("X"):
        return -1
    if " " not in board:
        return 0
    scores = []
    for i in range(9):
        if board[i] == " ":
            board[i] = "O" if is_max else "X"
            scores.append(minimax(not is_max))
            board[i] = " "
    return max(scores) if is_max else min(scores)
def best_move():
    best = -math.inf
    move = 0
    for i in range(9):
        if board[i] == " ":
```

```
            board[i] = "O"
            score = minimax(False)
            board[i] = " "
            if score > best:
                best = score
                move = i
    return move
def display():
    print(board[0:3])
    print(board[3:6])
    print(board[6:9])
for turn in range(9):
    display()
    if turn % 2 == 0:
        p = int(input("Enter position (1-9): ")) - 1
        if board[p] == " ":
            board[p] = "X"
        else:
            print("Invalid move")
            continue
    else:
        board[best_move()] = "O"
    if win("X"):
        display()
        print("Player X wins!")
        break
    if win("O"):
```

```
        display()
        print("Computer O wins!")
        break
else:
    display()
    print("Draw!")
```



RESULT

Thus, the **Minimax algorithm for gaming** was successfully implemented in Python, and the computer was able to select the optimal move.

CONCLUSION

The Minimax algorithm helps an AI player make the best possible decision by considering the possible moves of both players. It is commonly used in games such as **Tic Tac Toe, Chess, and Checkers**.